Spot a Keyword

For the next seven chapters you build ELIZA, the first chatbot. Before it can answer, it has to notice the word in your sentence that it knows something about.

Marcus Birkenkrahe

COSC 1210 · CS I · Session 1 of 2 · Augustana University

Fall 2026 · See also: Downey, *Think Python* 3e, ch. 7

Plan: seven chapters, one chatbot

Chapter	ELIZA learns to ...
7 Iteration and search	**spot a keyword, give a first reply**
8 Strings, regular expressions	turn "I am X" into "Why are you X?"
9 Lists	hold many rules and replies, remember
10 Dictionaries	swap "my" for "your", count what it does
11 Tuples	rank its rules, print a transcript
12 Text analysis and generation	find its blind spots, generate replies
13 Files and databases	keep its script and memory on disk

ELIZA, 1966

- You talked to ELIZA in week 2. Its best-known script, DOCTOR, imitates a therapist who turns your words back into questions
- Written by Joseph Weizenbaum at MIT, 1964–1966
- The program was written in **MAD-SLIP**, a list-processing extension of the language MAD, on an IBM 7094
- The scripts were written as nested lists that look like Lisp – the program itself was not Lisp
- The original source code was found in Weizenbaum's papers at MIT in 2021

How ELIZA answers

1. Look for a **keyword** in the sentence
2. Pick the rule that belongs to the keyword
3. Build the reply from the rule (and from pieces of your sentence)
4. No keyword? Say something neutral: "Please go on."

You type	Keyword	ELIZA answers
My mother is always busy.	mother	Tell me more about your family.
I had a strange dream.	dream	What does that dream suggest?
The weather is nice.	(none)	Please go on.

Try these sentences: `masswerk.at/elizabot`

Today is step 1: find the keyword. That takes loops.

Loop over a string

→ **Switch to editor · code-along 1**

```
for letter in "Why?":
    print(letter, end=" ")
```

```
W h y ?
```

The **loop variable** `letter` holds one character per pass. `end=" "` replaces the line break with a space.

Four answers. ELIZA needs **one**: is it a question or not?

How could you do that?

Is the sentence a question? Check every letter:

```
for letter in "Why?":
    if letter == "?":
        print(True)
    else:
        print(False)
```

```
False
False
False
True
```

One answer: print becomes return

→ **Switch to editor · TRY (2 min) · predict, then run**

```
def has_question_mark(sentence):
    for letter in sentence:
        if letter == "?":
            return True
        else:
            return False

print(has_question_mark("Why?"))
```

```
False
```

How would you fix it?

Scaffold `print` as the first line of the loop body:

```
    for letter in sentence:
        print("checking", letter)
```

```
checking W
False
```

→ The loop checks `"W"` and nothing else. The `else` runs `return False` – and `return` ends the whole function, and the loop with it.

The fix: "not found" goes after the loop

```
def has_question_mark(sentence):
    for letter in sentence:
        if letter == "?":
            return True
    return False

print(has_question_mark("Why?"))
print(has_question_mark("I am tired."))
```

```
True
False
```

- One `"?"` is enough: `return True` stays in the loop
- "No question mark" is only true after you have looked at *every* letter: `return False` goes after the loop
- `print` does not end anything; `return` does
- Question: what if you indent `return False` under the `if` instead of the `for`? What do you get, and why?

Count with a loop

→ **Switch to editor · code-along 2**

One more word than there are spaces:

```
def count_words(sentence):
    count = 1
    for letter in sentence:
        if letter == " ":
            count = count + 1
    return count

print(count_words("My mother is always busy."))
```

```
5
```

- count is a **counter**: **initialized** before the loop, updated inside it
- Short form: `count += 1` (**augmented assignment**), also `-=`, `*=`
- TRY (2 min): remove `count = 1`. What happens?

```
UnboundLocalError: local variable
'count' referenced before assignment
```

→ To update count, Python needs its old value – there is none.

Question: where does count_words go wrong?

What do these calls return? (The first one has two spaces.)

```
print(count_words("Hello  there"))
print(count_words(""))
```

```
3
1
```

→ Both wrong: the function counts spaces, not words. In chapter 9, `split` does it properly.

Is the word in the sentence?

→ **Switch to editor · code-along 3**

```
print("mother" in "My mother is always busy.")
print("father" in "My mother is always busy.")
print("mother" in "Mother is always busy.")
```

```
True
False
False
```

With strings, `in` tests for a **substring**. It is case-sensitive: `"Mother"` is not `"mother"`.

Lower the sentence first:

```
def mentions(sentence, word):
    return word in sentence.lower()

print(mentions("Mother is always busy.",
               "mother"))
print(mentions("Mother is always busy.",
               "Mother"))
print(mentions("Mother is always busy.",
               "MOTHER".lower()))
```

```
True
False
True
```

Why `False` in the middle?

Lower both sides

- `mentions` lowers only the sentence. `"Mother"` keeps its capital M and can never match
- The third call works because the caller lowers the word – but then every caller has to remember it
- Let the function do the work: lower both sides

```
def mentions(sentence, word):
    return word.lower() in sentence.lower()

print(mentions("Mother is always busy.", "mother"))
print(mentions("Mother is always busy.", "Mother"))
print(mentions("Mother is always busy.", "MOTHER"))
```

```
True
True
True
```

`lower()` is a **method**: a function called on a value. The comparison is already `True` or `False` – return it directly.

in finds letters, not words

→ **Switch to editor · TRY (2 min) · predict, then run**

```
print(mentions("The pillow smothered the sound.", "mother"))
print(mentions("I know.", "no"))
print(mentions("I like computers.", "computer"))
```

```
True
True
True
```

- "mother" is inside "smothered", "no" inside "know" – wrong
- "computer" inside "computers" – here it helps
- ELIZA lives with this until chapter 8: regular expressions can match whole words

Keywords belong in a file

→ **Switch to editor · code-along 4 · setup cell, run once**

```
open("keywords.txt", "w").write("""sorry
remember
dream
computer
mother
father
sister
brother
always
because
""")
```

You learn later how this works.

```
file = open("keywords.txt")
print(repr(file.readline()))
print(repr(file.readline()))
```

```
'sorry\n'
'remember\n'
```

- open returns a **file object**; `readline` reads one line
- Every line ends with a **newline** `\n`; `repr` shows it
- The file object remembers where it stopped

Read every line: `for` and `strip`

```
for line in open("keywords.txt"):
    print(line)
```

```
sorry

remember

dream
...
```

`\n` at the end of the line, plus the line break of `print`

```
for line in open("keywords.txt"):
    word = line.strip()
    print(word, end=" ")
```

```
sorry remember dream computer mother
father sister brother always because
```

`strip()` removes whitespace – spaces, `\n` – at both ends

Can you put the pieces together?

For each keyword in the file, check whether it is in the sentence:
`show_keywords(sentence)`

Which keywords are in the sentence?

```
def show_keywords(sentence):
    for line in open("keywords.txt"):
        word = line.strip()
        if word in sentence.lower():
            print(word)

show_keywords("My mother says I always "
              "dream about computers.")
```

```
dream
computer
mother
always
```

The keywords come out in the order of the **file**, not of the sentence.

Which one should ELIZA answer? That is the next question.

Practice: how many keywords?

→ **Switch to editor · 10 min, then 5 min discussion**

Write `count_keywords(sentence)`: it *returns* how many keywords from `keywords.txt` appear in `sentence` (ignore case). Use a counter.

```
>>> print(count_keywords("My mother and my father never talk."))
2
>>> print(count_keywords("Sorry, I forgot."))
1
>>> print(count_keywords("The weather is nice."))
0
```

Solution: how many keywords?

```
def count_keywords(sentence):
    count = 0
    for line in open("keywords.txt"):
        word = line.strip()
        if word in sentence.lower():
            count += 1
    return count
```

```
2
1
0
```

- `def`: a function we can test on many sentences – it *returns* the number
- `count = 0`: before the loop – no keyword found yet
- `for line in open(...)`: one line of the file at a time
- `strip()`: without it, `"mother\n"` is never in the sentence
- `in` + `lower()`: search the sentence – `"Sorry"` matches `"sorry"`
- `count += 1`: inside the `if` – count only what is found
- `return count`: *after* the loop – every keyword checked

Checkout

1. Why must `return False` in `has_question_mark` come after the loop?

→ "Not found" is only true after every character was checked. Inside the loop, the `else` returns after the first one.

2. Why does `print(line)` in a loop over a file print empty lines?

→ Each line ends with `\n`, and `print` adds another line break. Use `line.strip()`.

Summary

- A `for` loop over a string visits one character per pass; the **loop variable** holds the current one
- A `return` inside a loop ends the function at once. "Not found" goes after the loop
- A **counter** is initialized before the loop and updated inside it; `count += 1` is short for `count = count + 1`
- `in` tests whether a string is a **substring** of another. It is case-sensitive – use `lower()` – and it also finds a word inside another word
- `open` returns a **file object**; `readline` reads one line, a `for` loop reads all of them. Each line ends with `\n`; `strip()` removes it

Glossary: terms from session 1

#	Term	Meaning
1	chatbot	Program that converses in natural language
2	keyword	Word that tells ELIZA which rule to use
3	loop variable	Variable that takes a new value in each pass
4	counter	Variable that counts, e.g. `count += 1`
5	initialize	Give a variable its first value
6	augmented assignment	Update in place, e.g. `+=`
7	substring	String that is part of another string
8	method	Function called on a value, e.g. `s.lower()`
9	file object	What `open` returns; reads a file line by line
10	newline	Character `\n` that ends a line

Next: the first reply

`show_keywords` prints every keyword it finds. ELIZA needs exactly one.

- A **search** that stops at the first keyword
- ELIZA's first replies – and a fallback for everything else
- Tests written into the docstring